

Cell-Mediated, Not Terminal: A Formalization of the LLM-as-Terminal-Producer Anti-Pattern in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

...

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone anti-pattern, the failure mode in which LLM outputs flow directly to substrate as terminal artifacts without rule-governed transformation — a configuration that violates the AI-as-substrate-mediator commitment (§4.1, §4.2, §4.4) at the *output layer* — so downstream work has an independently citable target for the failure mode and for the architectural correction it requires.

Abstract

The CKS pattern's AI-as-substrate-mediator commitment requires that LLM operations on the substrate are bounded by orchestration rules at every architectural layer — when the LLM is invoked, when its inputs are assembled, and when its outputs flow back to substrate state. The output-layer requirement is the one most easily missed in 2024–2026 generative-AI deployments, where LLM-produced text, structured records, and decisions commonly flow directly into knowledge bases, content systems, and coordination stores without intermediary processing. This note formalizes that configuration as a standalone anti-pattern — *LLM-as-terminal-producer* — defined by four operational components: direct LLM-output-to-substrate write paths; no rule-governed transformation between generation and write; no cell-mediated validation of outputs; and no architectural intermediary between LLM generation and substrate state. The note identifies the CKS commitments violated, traces the failure mode it produces (most distinctively, hallucination injection into substrate as authoritative content), specifies the architectural correction, distinguishes the anti-pattern from four adjacent legitimate patterns, and provides an operational test with three sharpening properties for whether a deployment exhibits it. LLM-as-terminal-producer is the second of three foundational A1.04 anti-patterns differentiated by architectural failure point: A3.11 fails at the decision-making layer (LLM autonomously decides what to do); this note's anti-pattern fails at the output layer (LLM output becomes substrate state without rule-mediation); A3.13 fails at the authority layer (LLM-produced content becomes substrate-authoritative).

1. Why the anti-pattern needs to be formalized as standalone

The AI-as-substrate-mediator commitment is the operational hinge of the CKS pattern: the substrate handles what is human-governed and auditable, and the LLM handles high-dimensional reasoning under orchestration rules. The commitment does its work at three architectural points — invocation, input assembly, and the flow of outputs back to substrate state. Violations at each point are architecturally distinct. The autonomous-agent failure (A3.11) is the decision-layer failure: the LLM decides what to do without rule-governance. The terminal-producer failure (this note) is the output-layer failure: the LLM's outputs flow to substrate without rule-governed transformation between generation and write. The source-of-truth failure (A3.13) is the authority-layer failure: LLM-produced content becomes substrate-authoritative even where rule-mediation existed at earlier layers.

The output-layer failure is operationally common in 2024–2026 because generative AI is positioned as producing artifact-style outputs — drafted documents, structured records, conversational responses, multi-modal content — that visually appear "finished." Many AI products route LLM-generated content directly into knowledge bases, content systems, or coordination stores without intermediary processing. Deployments that adopt the CKS substrate may inherit this routing pattern without recognizing that direct LLM-output-to-substrate write paths violate the mediator commitment specifically at the output layer. The violation is not detectable by inspecting whether LLM invocations are governed (they may be) or whether attribution is recorded (it may be); only by inspecting whether rule-governed transformation operates between generation and substrate write.

A3.11 and A3.12 may operate independently or compound: a deployment may have rule-mediated invocations driven by workflow logic but admit LLM outputs to substrate as terminal artifacts (A3.12 without A3.11), or have an autonomous LLM whose decisions invoke rule-mediated cells with rule-transformed outputs (A3.11 without A3.12), or have both at once.

2. The anti-pattern, defined precisely

A deployment exhibits **LLM-as-terminal-producer** if all four of the following operational components are present:

1. **Direct LLM-output-to-substrate write paths.** The deployment configuration includes paths in which LLM outputs flow to substrate writes without architectural intermediary. The LLM generates text, structured records, decisions, or multi-modal content, and that content becomes substrate state.
2. **No rule-governed transformation between generation and write.** No orchestration rule specifies what transformation applies to LLM outputs before they become substrate state. Rules that exist may govern when the LLM is invoked or what inputs it receives, but they do not govern what the output looks like, what structure it must have, or whether it should be admitted to substrate at all.
3. **No cell-mediated validation or processing of LLM outputs.** No cell mediates between LLM generation and substrate write to validate the output against substrate constraints — structural (schema, format, relationships), semantic (consistency with existing substrate content), or factual (presence of grounding in inputs). The output is admitted as-is.
4. **No architectural intermediary between LLM generation and substrate state.** The deployment treats LLM generation as operationally identical to substrate write. The architectural pattern is *LLM-output* → *substrate-write*, with no intermediating cell — rather than *LLM-output* → *cell-rule-transformation* → *substrate-write*, which is what the mediator commitment requires at the output layer.

A deployment exhibiting all four components fully exhibits the anti-pattern. A deployment exhibiting only some — for example, rule-governed invocation and input assembly but an unmediated output path — partially exhibits it; the failure emerges at whichever layer rule-mediation fails. The four components specify the failure architecturally, independent of which output type (text, structured record, decision, multi-modal) is involved and independent of which substrate technology hosts the writes.

3. Which CKS commitments are violated

LLM-as-terminal-producer is operationally distinguishable from sibling A1.04 anti-patterns by which commitments it violates and at which architectural point.

Directly violated. The mediator commitment (§4.1, §4.2) is directly violated at the output layer — outputs leave the LLM and enter substrate without rule-mediated transformation. Property B of the mediator role (LLM writes under orchestration rules) is the specific operational property the

anti-pattern fails: terminal-producer writes are LLM-output-determined, not rule-determined; what reaches substrate is what the LLM generated, not what an orchestration rule transformed the LLM's generation into. The orchestration-rule authoring moment §3.3 names as the canonical governance moment is bypassed at the output layer; rules may govern invocation and inputs, but not transformation of outputs into substrate state.

Partially compromised. Property E (LLM operations recorded with attribution) is partially compromised. Attribution may exist — the substrate may record that an LLM produced the content — but the rule-governance component of attribution is absent, because no rule governed the transformation. Path retraceability (§3.1) loses the moment at which an orchestration rule shaped the substrate write, leaving a trail in which the rationale for substrate state is "the LLM produced this" rather than "rule R transformed inputs I into substrate state S."

Operationally implicated. Property D (LLM does not exercise authority) is operationally implicated even where the LLM does not exercise *decision* authority — which is the architectural failure that defines A3.11. Terminal-producer operation makes the LLM's *output* operationally authoritative for substrate content, since what reaches substrate is what the LLM generated. The substrate-as-source-of-truth structure (§11.3) carries content shaped by LLM generation rather than by rule-governed coordination.

Extended-implicated. The human-governed commitment (§3.1, §3.3) is extended-implicated: humans cannot govern substrate content effectively when LLM outputs are terminal because the rule-mediation mechanism through which governance reaches operational behavior does not apply at the output layer. The override right remains exercisable post-hoc; the architectural prevention path is gone. The determinism contract is similarly extended-implicated — LLM output non-determinism is permitted within rule-governed cells, but the architectural commitment that the determinism boundary sits at substrate operations fails when LLM outputs flow directly to substrate without rule-governed bounding.

The cascade across the mediator role at the output layer, governance reach at the output layer, and substrate-as-source-of-truth integrity is what makes the anti-pattern operationally significant beyond a narrow violation of Property B alone.

4. The failure mode

LLM-as-terminal-producer produces deployments in which substrate state is shaped by LLM generation rather than rule-governed cell processing.

The most distinctive harm is **hallucination injection into substrate as authoritative content**. LLMs occasionally produce content not grounded in their inputs; under cell-mediated processing, an orchestration rule could include validation logic that rejects ungrounded outputs (e.g., "cell output must reference at least one substrate fact present in the cell's input"). Under terminal-producer operation, hallucinations flow directly into substrate without rule-validation; substrate carries LLM-fabricated content architecturally indistinguishable from validated content. This harm pattern distinguishes A3.12

from A3.11: A3.11's distinctive harm is autonomous decision divergence; A3.12's distinctive harm is generation errors becoming substrate errors directly.

Three closely related consequences follow. **Structural mismatch** admits content that does not match substrate's schema, format, or addressing commitments, sometimes silently. **Semantic content becoming coordination state** admits LLM-generated narratives, descriptions, and decisions as substrate state as-is, so substrate carries content shaped by the LLM's generation patterns rather than by the rule-governed shaping the cell's rules were meant to perform. **Error-direct-flow** admits incorrect content, missing required elements, format issues, and content drift across invocations; there is no architectural step at which errors are detected before substrate state is updated.

Two structural consequences extend the failure. **Recovery becomes post-hoc modification**: when terminal-producer outputs are subsequently identified as erroneous, recovery requires humans exercising the modify right to correct content that should have been corrected at the output layer. Recovery is operationally feasible but architecturally inconsistent — the architectural prevention path was absent. **The anti-pattern compounds with A3.13**: terminal-producer outputs that downstream operations treat as authoritative produce A3.13. The progression is operationally common because nothing in substrate state distinguishes content admitted under rule-mediation from content admitted as terminal LLM output once both sit in substrate. Multi-modal LLM outputs (text, images, audio, embeddings) flowing directly to substrate extend the failure across content types; structural mismatch and hallucination risks compound across modalities, since validation logic applicable to text outputs may not exist for embeddings or images.

5. The architectural correction

The correction operates through three commitments together: cell-mediated LLM output processing per the mediator commitment, rule-governed transformation at the output layer per Property B, and the rule-authoring-as-governance-moment principle (§3.3) applied to output transformation specifically.

Cell-mediated LLM output processing. LLM outputs flow through rule-governed cells before becoming substrate state. The cell processes the LLM output according to its orchestration rule; the cell may transform, validate, filter, restructure, or reject the output; the cell writes the processed result to substrate as a Property B substrate write.

Rule-governed transformation at the output layer. The orchestration rule for each cell that processes LLM outputs specifies the transformation that applies. A rule may specify structural requirements (the output must populate specific fields), semantic requirements (the output must reference substrate facts present in the cell's input), validation requirements (the output must pass specific format or grounding checks), and attribution requirements (the substrate write must record provenance per the path-retraceability commitment, including which rule transformed which input into which substrate state).

Architectural intermediary between generation and substrate. The deployment configuration interposes a cell — not a logging hook, not a post-hoc audit trail, but a cell whose orchestration rule specifies the transformation — between LLM generation and substrate write. The intermediary is what makes the difference between mediator operation and terminal-producer failure architecturally identifiable.

Validation against substrate constraints, and decomposition of generative patterns. Cells processing LLM outputs include validation logic for substrate structural, semantic, and factual constraints; hallucinations, structural mismatches, and content drift are detected and rejected before substrate write rather than corrected after. Where deployments use generative-AI patterns (LLM-generated documents, AI-generated content, conversational AI outputs), the architectural correction decomposes these into cell-mediated processing — the LLM generates *within* a cell; the cell processes; the processed output becomes substrate state. The generative pattern is preserved as labor; the rule-governed shaping is preserved as architecture. Re-architecting a deployment that has drifted into terminal-producer operation can be substantial — identifying every direct write path, designing the orchestration rule that governs the transformation at each path, and making validation logic explicit.

6. What the anti-pattern is NOT

Four adjacent legitimate patterns are commonly conflated with LLM-as-terminal-producer; the architectural distinction sits at whether rule-governed transformation operates at the output layer.

Not cell-mediated LLM outputs that are rule-transformed before substrate write. Cells where LLM outputs are processed by orchestration rules — rule-specified transformations, validations, and attributions applied before substrate write — are the legitimate pattern the mediator commitment requires. The anti-pattern is the configuration where outputs flow directly to substrate; cell-mediated rule-transformed outputs satisfy the architecture.

Not LLM as cell consultation. Cells consulting an LLM as part of their rule-governed execution — the LLM as a high-dimensional reasoning component the cell invokes under its orchestration rule — operate within the mediator commitment. The cell's output to substrate is rule-shaped, even though the LLM contributed to the cell's intermediate computations.

Not LLM as drafting assistant where humans review before applying. LLMs that produce drafts which humans review and apply to the substrate are legitimate; the human exercising the modify right under direct authority is the rule-mediation mechanism in this case. The failure is different: drafting that humans review is legitimate; LLM outputs flowing to substrate without review or rule-mediation is not.

Not generative AI outputs that humans choose to record. Humans may use generative AI to produce content and choose to record selected outputs into substrate by exercising the modify right directly. Human-directed recording is legitimate; automatic flow of LLM outputs to substrate without human direction or rule-mediation is the failure.

The four distinctions share a common structure: legitimate adjacents preserve rule-mediation through cell-orchestration-rule transformation or through direct human authority; the anti-pattern is configurations where neither is present.

7. Operational test

A deployment exhibits LLM-as-terminal-producer if any of the following are true at any time during the deployment's existence:

1. LLM outputs flow to substrate writes without rule-governed transformation between generation and write.
2. The deployment lacks orchestration rules that specify transformations, validations, or attributions applied to LLM outputs before substrate write.
3. Cells do not mediate between LLM generation and substrate write; the deployment has direct LLM-output-to-substrate paths.
4. Substrate state may include LLM-generated content (text, structured records, decisions, multi-modal artifacts) without architectural indication of cell-rule-transformation.

Three sharpening properties make the test operationally usable for deployment review.

Rule-transformation-presence. Trace each LLM-output path to substrate; the absence of an orchestration rule that specifies the transformation applied at the path indicates the anti-pattern.

Cell-mediation-at-output. Examine the architectural relationship between LLM generation and substrate state changes; direct paths — paths in which LLM generation is the substrate write event without an intermediating cell — indicate the anti-pattern.

Hallucination-substrate-isolation. Simulate ungrounded outputs (outputs whose content is not present in or derivable from the cell's substrate inputs) at LLM-affecting-substrate paths; outputs that flow to substrate without rejection indicate the anti-pattern. The third test is the most operationally distinctive of the three because it isolates the architectural failure to the output layer specifically — invocation governance, input-assembly governance, and post-hoc audit may all be present, and the anti-pattern is still detectable through ungrounded-output admission.

A deployment that fails any of (1)–(4) and any of the three sharpening properties exhibits the anti-pattern; the architectural correction in §5 specifies the operational changes required.

The one-sentence test. *If a deployment's LLM outputs flow directly to substrate as terminal artifacts — generated text becoming substrate content, generated structured records becoming substrate entries, generated decisions becoming substrate state — without rule-governed cell transformation between LLM generation and substrate write, the deployment exhibits LLM-as-terminal-producer; the architectural commitment to AI-as-substrate-mediator fails specifically at the output layer through Property B (LLM writes under orchestration rules), with the rule-authoring-as-governance moment*

bypassed at the output layer and Property E attribution lacking the rule-governance component path retraceability requires.

8. Why naming this anti-pattern as standalone matters

Implementations under pressure to deliver AI products with generative-AI capabilities consistently default to terminal-producer routing because generative AI is positioned as producing artifact-style outputs that appear finished. The drift is steady, and audiences understand "we use generative AI to produce content for our knowledge base" as standard engineering rather than as the architectural choice it is. The harm pattern that follows is distinctive — hallucination injection into substrate as authoritative content — and is separable from the harm patterns of A3.11 (autonomous decision divergence) and A3.13 (substrate-authoritative LLM content). The architectural failure point is also distinct: A3.11 fails at the decision-making layer, A3.12 (this note) at the output layer, A3.13 at the authority layer; the three may operate independently or compound. Naming each anti-pattern separately is what allows the architectural failure point of a given deployment to be identified, and the architectural correction to be applied at that point rather than across the deployment uniformly. Subsequent work that adopts, extends, composes, or argues against the CKS pattern should treat *LLM-as-terminal-producer* in the sense formalized here — the deployment configuration in which LLM outputs flow directly to substrate as terminal artifacts without rule-governed transformation between generation and write — and should name any other use of the term as a different concept.

...

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Cell-Mediated, Not Terminal: A Formalization of the LLM-as-Terminal-Producer Anti-Pattern in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.